I have 3 years of experience as a Quality Analyst in a product-DIGITAL workspace.

In my current role, I've been involved in both manual and automation testing.

I've worked across web, desktop, and mobile applications, and have also tested enterprise dashboards for data validation,

On the automation side, I've written Python scripts using Selenium and Pytest to validate dashboard widgets

Also i have created the generalized script's using robot framework for our funcanalitys.

We follow Agile methodology, so I'm used to working in sprints

Good morning first Thank you for giving me the opportunity to introduce myself,

I have 3 years of experience as a Quality Analyst in a product-based company Digitate inside tcs, Our product is primarily a monitoring tool called Ignio which is a monitoring and Inc management tool. It

consists of several sub products. Among them I am working on a product called DIGITAL workspace (TCS).

In my current role, I've been involved in both manual and automation testing.

I've worked across web, desktop, and mobile applications, and have also tested enterprise dashboards funcnalitis and data accuracy, ensuring the data is accurate and aligned with business requirements.

On the automation side, I've written Python scripts using Selenium and Pytest to validate dashboard widgets, which has helped improve our testing efficiency.

We follow Agile methodology, so I'm used to working in sprints, And I while testing identified critical defects and worked closely with the team to resolve them before release, contributing to a stable and reliable product.

I always work to ensure a smooth user experience and I'm open to learn new tools and technologies to enhance the testing process.

Question and topics need to be covered

Manual testing types with short definitions:

1.Integration Testing: Testing combined modules to ensure they work together.

2. Unit Testing: Testing individual components or modules.

3.System Testing: Testing the complete application as a whole.

4.Sanity Testing: **Sanity Testing** is a focused testing approach performed after minor changes or bug fixes to ensure the affected functionalities are working as expected.

5.Smoke Testing: **Smoke Testing** is a high-level testing process conducted to verify that the critical functionalities of a new build are working and the application is stable for further testing.

6. Regression Testing: Verifying new changes haven't affected existing features.

7. Functional Testing: Checking if features meet functional requirements.

8. Non-functional Testing: Testing performance, usability, and reliability.

9. User Acceptance Testing (UAT): Ensuring the application meets end-user needs.

11. Exploratory Testing: Testing without predefined test cases, exploring the app.

12. Compatibility Testing: Ensuring the app works on different devices, OS, and browsers.

13. Security Testing: Checking the app's security measures.

14. Performance Testing: Validating speed, scalability, and stability.

---

2. abstract class and where did you implement in your project

from abc import ABC, abstractmethod

class BasePage(ABC): @abstractmethod def click_element(self, locator): pass

```
@abstractmethod
def enter_text(self, locator, text):
    pass
```

class LoginPage(BasePage): def click_element(self, locator): print(f"Clicking on {locator}")

```
def enter_text(self, locator, text):
    print(f"Entering '{text}' into {locator}")
```

login = BasePage() # ❌ This will raise an error (can't instantiate abstract class)

login_page = LoginPage() login_page.enter_text("username_field", "test_user")

3. list comprehension in python

```
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]

newlist = [x for x in fruits if "a" in x]

print(newlist)
```

4. decorators in python

```
def my_decorator(func):
    def wrapper():
        print("Before function execution")
        func()
        #print("After function execution")
    return wrapper


@my_decorator  # Applying the decorator
def say_hello():

    print("Hello, World!")
```

say_hello()

✅ Method 1: Using a Property with a Custom Decorator

python Copy Edit

```python
def uppercase_decorator(func):
    def wrapper(self, value):
        func(self, value.upper())
    return wrapper

class Person:
    def __init__(self, name):
        self.name = name

    @property
    def name(self):
        return self._name

    @name.setter
    @uppercase_decorator
    def name(self, value):
        self._name = value

# Example
p = Person("gokul")
print(p.name)  # Output: GOKUL
```

✅ Version 2: Using a Decorator to Enforce Uppercase Check

If you want a reusable decorator to enforce uppercase validation:

python Copy Edit

```python
def validate_uppercase(func):
    def wrapper(self, value):
        if not value.isupper():
            raise ValueError("Value must be in uppercase.")
        return func(self, value)
    return wrapper

class Person:
    def __init__(self, name):
        self.name = name

    @property
    def name(self):
        return self._name

    @name.setter
    @validate_uppercase
    def name(self, value):
        self._name = value
```

5. Find duplicates in list or find unique number and get the total numbers = [1, 2, 3, 4, 2, 3, 5, 6, 1, 7, 8, 1]

duplicates = {}

for num in numbers:

if num in duplicates:

duplicates[num] += 1 # Increment count if already in dictionary

else:

duplicates[num] = 1 # Initialize count

```
# Print only the numbers that appear more than once
for key in duplicates:
    if duplicates[key] > 1:
        print(f"{key} appears {duplicates[key]} times")
```

6. Fixtures in pytest and how to pass the argument

7. agile meetings what are all the event of meeting that you attended

1. **Daily Stand-up**
2. **Sprint Planning**
3. **Sprint Review**
4. **Sprint Retrospective**
5. **Backlog Refinement**

8. Xpath

9. Psycopg2 setup in python

python Copy Edit

```python
import psycopg2

class PostgresDB:
    def __init__(self, host, dbname, user, password):
        self.database = psycopg2.connect(
            host=host,
            database=dbname,
            user=user,
            password=password
        )
        self.cur = self.database.cursor()

    def execute_query(self, query):
        self.cur.execute(query)
        return self.cur.fetchall()  # For SELECT queries only

    def close(self):
        self.cur.close()
        self.database.close()
```

✅ Example Usage:

```python
db = PostgresDB("localhost", "your_db", "your_user", "your_password")

result = db.execute_query("SELECT * FROM employees;")
print(result)

db.close()
```

✅ Here's how to handle **all types of queries** in a clean way:

```python
def execute_query(self, query):
    self.cur.execute(query)
    if query.lower().startswith("select"):
        return self.cur.fetchall()
    else:
        self.database.commit()
        return "Query executed."
```

🔍 Explanation:

- `SELECT`: You **fetch and return results**.
- `UPDATE`, `DELETE`, `INSERT`: You **commit the transaction** and return a message.
- `query.lower().startswith("select")`: Checks if the query is a SELECT (case-insensitive).

10. API request codes

12. Types of wait in Selenium and what are the problem or issue you faced in your code

13. Joins in sql

14. Marks in pytest

In **pytest**, `marks` are used to add **metadata or labels** to test functions so you can:

- **Group**, **filter**, or **configure** them,
- **Skip** or **xfail** (expected to fail) tests,
- Use them for **parameterization**, or
- Apply **custom behavior** during test runs.

✅ **Common Pytest Marks:**

Mark	Description	Example
`@pytest.mark.skip`	Skips the test unconditionally	`@pytest.mark.skip(reason="Not ready")`
`@pytest.mark.skipif`	Skips the test **if a condition is True**	`@pytest.mark.skipif(sys.platform == "win32", reason="Only for Linux")`
`@pytest.mark.xfail`	Marks test as expected to fail	`@pytest.mark.xfail(reason="Bug #123")`
`@pytest.mark.parametrize`	Runs the same test function with different data	`@pytest.mark.parametrize("x, y", [(1, 2), (3, 4)])`

15. What module do you use to write a test case

16. method overloading and overriding in python and write a simple program

Method Overloading

Defining multiple methods with the same name but different parameters.

```
class MathOperations:
   def add(self, a, b=0, c=0):  # Default arguments used for
overloading
      return a + b + c

math = MathOperations()
print(math.add(2))       # Output: 2
print(math.add(2, 3))    # Output: 5
print(math.add(2, 3, 4))  # Output: 9
```

Method Overriding

Redefining a method from the parent class in the child class.

```
class Parent:
    def show(self):
        print("This is the parent class")

class Child(Parent):
    def show(self):  # Overriding the parent method
        print("This is the child class")

obj = Child()
obj.show()  # Output: This is the child class
```

17. How do you run the pytest and its key words

18. What is args and kwags in python

```
def mix_example(*args, **kwargs):
    print("Positional arguments:", args)
    print("Keyword arguments:", kwargs)
mix_example(1, 2, 3, name="Alice", age=25)
```

19. Basic queries for all types in SQL and explain

20. What are the Git comments that you used in your project

21. How to delete the branch in git

22. mutable and Immutable's python

23. Difference between close() and quit() methods in Selenium?

24. How to handle multiple windows in Selenium?

Second Highest number in python

25. How many types of XPath are there?

26. How to delete a branch in git
27. Print the occurrence of a given string if string is “dineshkumars” we need to print all charaters and how may time it is there in a string
28. Print the unique characters in a given 2 string

```
a="skjdhdsda"
b="dslfihoaug"
mylist=[]

def unique(a):
   for i in a:

      if i not in mylist:

         mylist.append(i)

unique(a)
unique(b)
print(mylist)
```

or

```
a = 'alnnwnewidd'
b = 'adflnwlgfwff'
d=[]
c=[d.append(a) for a in a+b if a not in d]
print(''.join(d))
```

o/p -> alnweidfg

29. Get a top 4 highest persons and if we have 2 people getting same 1 hiest salary also we should print both Salarys

SELECT * FROM employee WHERE salary IN (SELECT DISTINCT salary FROM employee ORDER BY salary DESC LIMIT 4);

30. To get array input using list comprehension

arr = [int(x) for x in input().split()]

31. : How do you verify in Selenium that the password field masks user input?

A: Locate the password input and check if its `type` attribute is `"password"` using `get_attribute("type")`.

If it returns `"password"`, the field is masking input correctly.

32. What is the `monkeypatch` fixture in Pytest?

The `monkeypatch` fixture in **pytest** is a built-in utility used to **safely modify or replace attributes, dictionaries, environment variables, and objects during test execution**. It's especially useful for isolating parts of your code in unit tests, such as mocking out dependencies.

Example 1: Replacing a function

python Copy Edit

```python
# app.py
def get_data():
    return "real data"
```

python Copy Edit

```python
# test_app.py
from app import get_data

def test_get_data(monkeypatch):
    def mock_get_data():
        return "fake data"

    monkeypatch.setattr("app.get_data", mock_get_data)
    assert get_data() == "fake data"
```

Example 2: Modifying environment variables

python Copy Edit

```python
import os

def get_api_key():
    return os.getenv("API_KEY", "default")
```

python Copy Edit

```python
def test_get_api_key(monkeypatch):
    monkeypatch.setenv("API_KEY", "testkey")
    assert get_api_key() == "testkey"
```

Example 3: Modifying dictionary values

python Copy Edit

```python
config = {"debug": False}

def is_debug_mode():
    return config["debug"]
```

python Copy Edit

```python
def test_debug_mode(monkeypatch):
    monkeypatch.setitem(config, "debug", True)
    assert is_debug_mode() is True
```

33. What is the purpose of `pytest.ini` or `pyproject.toml` file?

oth `pytest.ini` and `pyproject.toml` are **configuration files** used to customize the behavior of **pytest**, a popular Python testing framework. They serve a similar purpose but follow different formats and standards.

34. What is the difference between xfail and skip in pytest?

Tests with the xfail mark are run but expected to fail, while tests with the skip mark are not executed at all.

35. SQL Aggregate Functions

36. ✅ **How to manage test cases in JIRA:**
Test cases in JIRA can be managed using test management plugins like Zephyr, Xray, or TestFLO. These tools allow you to create a separate issue type for test cases, define test steps, expected results, and group them into test cycles for execution. You can link test cases to user stories and track the testing progress through dashboards and reports. Without plugins, you can still manage test cases by creating a custom issue type and adding necessary fields, but it will have limited functionality compared to using add-ons.

37. ✅ **What is Confluence in JIRA:**
Confluence is a documentation and collaboration tool integrated with JIRA, used to create, share, and manage project documents like requirements, test plans, meeting notes, and reports. It allows teams to embed live JIRA issues, create traceability between requirements and test cases, and maintain a project knowledge base.

Confluence enhances transparency and teamwork by enabling everyone to access and contribute to project documentation in one central location.

38. What is Acceptance testing?

Acceptance testing is a type of software testing performed by users or customers to validate whether a software application meets their requirements and is ready for deployment

Acceptance testing is a type of software testing performed to determine whether a system meets the specified business requirements and is ready for delivery to the end users or customers.

39. Absolute path: The whole path name required to access a location in the file system, starting from the root directory.

Relative path: Describes the location of a file or folder relative to the current working directory.

40. The primary mechanism for parameterization in `pytest` is the `@pytest.mark.parametrize` decorator.

✅ **Example with Actual Values:**

python Copy Edit

```python
import pytest

@pytest.mark.parametrize("x, y", [
    (2, 3),
    (4, 5),
    (6, 7)
])
def test_addition(x, y):
    assert x + y > 0
```

41. Different between raise and try and catch

Raise:

When an input doesn't meet certain criteria.

When a critical part of the program fails and you want to stop further execution.
To raise a custom exception in larger applications.

Try and Catch:
When you're performing operations that might fail at runtime (e.g., division, file handling, API calls).
When you want to handle errors gracefully without stopping the program.
When you're dealing with user input that may cause exceptions (e.g., converting input to integer).
When you're accessing external systems like databases, files, or networks that may be unavailable.
When you want to log errors or show a custom message instead of a crash screen.

42. Self and __init__ in python

✅ `self` Keyword in Python

🔹 What is `self`?

- `self` represents the **current instance of the class**.
- It is used to **access instance variables and methods**.

🔹 Why is it needed?

- Python does **not use** `this` like other languages (Java/C++).
- `self` allows each object to **keep its own data separate**.

🔹 Example:

python Copy Edit

```python
class Person:
    def __init__(self, name):
        self.name = name  # assigns the name to the current object

    def greet(self):
        print(f"Hello, my name is {self.name}")
```

✅ Constructor in Python (`__init__`)

🔹 What is it?

- `__init__` is a **special method** in Python classes.
- It is called **automatically when an object is created**.
- Used to **initialize object attributes**.

🔹 Real-time example:

python Copy Edit

```python
class Laptop:
    def __init__(self, brand, price):
        self.brand = brand
        self.price = price

l1 = Laptop("Dell", 50000)
print(l1.brand)   # Output: Dell
```

Here, `__init__` sets values for `brand` and `price` when you create a new `Laptop` object.

🔁 Summary Table:

Term	Meaning	Use Case
`self`	Refers to current object instance	Access or update object variables
`__init__()`	Constructor, runs when object created	Initialize values like name, age, etc.

43. Git Rebase

git rebase branch_name: Rebase the current branch onto another branch.

```
57
58 try:
59     file = open("sample.txt", "r")
60     content = file.read()
61     print(content)
62     file.close()
63
64     file = open("sample.txt", "a")
65     file.write("\n123")
66     file.close()
67
68 except Exception:
69     print("file is not present")
70
71     with open('sample.txt','w') as file:
72         file.write('456')
73
74
75 finally:
76     print('done')
77
```

Software Development Life Cycle (SDLC)

SDLC is a process to design, develop, and deliver high-quality software. It consists of the following phases:

1. **Requirement Analysis**: Understand user needs and document requirements.
2. **Design**: Create architecture and detailed design documents.
3. **Implementation**: Write and integrate the code.
4. **Testing**: Validate the software against requirements (QA and bug fixes).
5. **Deployment**: Release the software to production.

6. **Maintenance**: Provide updates, fixes, and enhancements.

Software Testing Life Cycle (STDLC)

STDLC focuses on testing and quality assurance within STDLC. It includes:

1. **Requirement Analysis**: Understand testable requirements.
2. **Test Planning**: Create a test strategy, estimate resources, and define test objectives.
3. **Test Case Development**: Write detailed test cases and scripts.
4. **Test Environment Setup**: Prepare hardware/software for testing.
5. **Test Execution**: Run test cases and log defects.
6. **Test Closure**: Analyze results, create a closure report, and finalize deliverables.

Severity vs Priority

◇ Severity

- **Definition**: Indicates the **impact** of the defect on the application's functionality.
- **Set By**: Usually the **Tester**
- **Types**:
    - **Critical/Blocker**: App crashes or critical function is broken
    - **Major**: Important functionality fails
    - **Minor**: Small UI issue, does not impact functionality
    - **Trivial**: Cosmetic issue like a spelling mistake

◇ Priority

- **Definition**: Indicates **how soon** the defect should be fixed.
- **Set By**: Usually the **Developer/Manager/Lead**
- **Types**:
    - **High**: Must be fixed immediately (e.g., login not working)
    - **Medium**: Can be fixed in the regular sprint
    - **Low**: Fix in the future (e.g., typo in help section)